

Redis Tutorial: From Beginning to Advanced

A complete, hands-on guide to learning Redis — the in-memory data structure store used for caching, sessions, queues, pub/sub, and more.

Table of Contents

1. [What is Redis?](#)
 2. [Installation](#)
 3. [Getting Started with redis-cli](#)
 4. [Strings](#)
 5. [Lists](#)
 6. [Sets](#)
 7. [Hashes](#)
 8. [Sorted Sets](#)
 9. [Key Expiration \(TTL\)](#)
 10. [Transactions](#)
 11. [Lua Scripting](#)
 12. [Pub/Sub Messaging](#)
 13. [Persistence \(RDB & AOF\)](#)
 14. [Using Redis from Code \(Python Example\)](#)
 15. [Common Use Cases](#)
 16. [Replication & Clustering Basics](#)
 17. [Security Basics](#)
 18. [Performance Tips](#)
 19. [Next Steps](#)
-

1. What is Redis?

Redis (REmote DIctionary Server) is an open-source, in-memory data structure store. It can be used as a:

- **Database** — stores data in memory for extremely fast reads/writes
- **Cache** — sits in front of a slower database to speed up repeated queries
- **Message broker** — via Pub/Sub or Streams
- **Session store** — for web applications

Key characteristics:

Feature	Description
Speed	Data lives in RAM, so operations are typically sub-millisecond
Data structures	Not just key-value — supports strings, lists, sets, hashes, sorted sets, streams, and more
Persistence	Optional disk snapshots (RDB) or append-only logs (AOF) so data isn't lost on restart
Single-threaded core	Simplifies concurrency, though modern Redis uses threads for I/O
Atomic operations	Every command is atomic, and multiple commands can be batched atomically

2. Installation

macOS (Homebrew)

```
brew install redis
```

```
brew services start redis
```

Ubuntu/Debian

```
sudo apt update
```

```
sudo apt install redis-server
```

```
sudo systemctl enable redis-server
```

```
sudo systemctl start redis-server
```

Docker (any OS — recommended for trying it quickly)

```
docker run -d --name my-redis -p 6379:6379 redis:latest
```

Verify it's running

redis-cli ping

Expected output: PONG

3. Getting Started with redis-cli

`redis-cli` is Redis's interactive command-line client.

redis-cli

You'll get a prompt like:

```
127.0.0.1:6379>
```

Try a few basics:

```
127.0.0.1:6379> SET greeting "Hello Redis"
```

OK

```
127.0.0.1:6379> GET greeting
```

"Hello Redis"

```
127.0.0.1:6379> DEL greeting
```

(integer) 1

```
127.0.0.1:6379> EXISTS greeting
```

(integer) 0

Useful general commands:

KEYS * # list all keys (avoid in production on large datasets)

DBSIZE # count of keys in current database

FLUSHDB # delete all keys in current database

FLUSHALL # delete all keys in all databases

TYPE key # returns the data type stored at a key

SELECT 1 # switch to database index 1 (Redis has 16 logical DBs, 0-15, by default)

4. Strings

The simplest data type — a key mapped to a string (which can also hold numbers or binary data).

SET user:1:name "Alice"

GET user:1:name

APPEND user:1:name " Smith" # appends to existing string

STRLEN user:1:name # length of the string

Numeric operations

SET counter 10

INCR counter # 11

INCRBY counter 5 # 16

DECR counter # 15

DECRBY counter 3 # 12

Set with expiration

SET session:abc "active" EX 60 # expires in 60 seconds

SET session:abc "active" PX 60000 # expires in 60000 milliseconds

Set only if not exists / only if exists

SET lock:job1 "locked" NX # only sets if key doesn't exist (useful for locks)

SET lock:job1 "locked" XX # only sets if key already exists

Multiple keys at once

MSET a 1 b 2 c 3

MGET a b c

5. Lists

Ordered collections of strings — implemented as linked lists, great for queues and stacks.

RPUSH tasks "task1" # push to the right (tail)

RPUSH tasks "task2" "task3"

LPUSH tasks "task0" # push to the left (head)

LRANGE tasks 0 -1 # get all elements (0 to -1 = start to end)

Result: "task0" "task1" "task2" "task3"

LLEN tasks # length of the list

LPOP tasks # remove and return leftmost element

RPOP tasks # remove and return rightmost element

LINDEX tasks 1 # get element at index 1

LSET tasks 0 "newtask" # set value at index 0

LTRIM tasks 0 1 # keep only elements in range 0-1

Blocking operations (useful for building queues/worker patterns):

BLPOP tasks 5 # block for up to 5 seconds waiting for an element

BRPOP tasks 0 # block indefinitely

6. Sets

Unordered collections of unique strings.

SADD tags "redis" "database" "nosql"

SMEMBERS tags # list all members

SISMEMBER tags "redis" # 1 (true) or 0 (false)

SCARD tags # count of members

SREM tags "nosql" # remove a member

Set algebra

SADD setA 1 2 3

SADD setB 2 3 4

SINTER setA setB # intersection -> 2 3

SUNION setA setB # union -> 1 2 3 4

SDIFF setA setB # difference -> 1

SPOP setA # remove and return a random member

SRANDMEMBER setA 2 # get 2 random members without removing

7. Hashes

Field-value pairs within a single key — perfect for representing objects.

HSET user:1 name "Alice" age 30 city "Paris"

HGET user:1 name # "Alice"

HGETALL user:1	# all fields and values
HMGET user:1 name city	# get multiple fields
HDEL user:1 city	# delete a field
HEXISTS user:1 age	# 1 or 0
HINCRBY user:1 age 1	# increment numeric field
HKEYS user:1	# all field names
HVALS user:1	# all values
HLEN user:1	# number of fields

8. Sorted Sets

Like sets, but each member has an associated **score**, keeping members ordered. Great for leaderboards, rankings, and priority queues.

ZADD leaderboard 100 "Alice"

ZADD leaderboard 85 "Bob"

ZADD leaderboard 95 "Carol"

ZRANGE leaderboard 0 -1 WITHSCORES # ascending order

ZREVRANGE leaderboard 0 -1 WITHSCORES # descending order

ZSCORE leaderboard "Alice" # get Alice's score

ZRANK leaderboard "Bob" # rank (ascending, 0-indexed)

ZREVRANK leaderboard "Bob" # rank (descending)

ZINCRBY leaderboard 10 "Bob" # increment Bob's score by 10

ZREM leaderboard "Carol" # remove a member

ZRANGEBYSCORE leaderboard 90 100 # members with scores 90-100

ZCOUNT leaderboard 90 100 # count of members in score range

9. Key Expiration (TTL)

Any key (regardless of type) can have an expiration.

SET session:xyz "data"

EXPIRE session:xyz 30 # expires in 30 seconds

TTL session:xyz # seconds remaining, -1 if no expiry, -2 if key doesn't exist

PERSIST session:xyz # remove the expiration, key lives forever

PTTL session:xyz # TTL in milliseconds

This is what makes Redis great for caching — data can automatically expire without manual cleanup.

10. Transactions

Redis transactions batch multiple commands so they run atomically, uninterrupted by other clients.

MULTI

SET balance:1 100

DECRBY balance:1 20

INCRBY balance:2 20

EXEC

- **MULTI** starts queuing commands.
- **EXEC** executes all queued commands atomically.
- **DISCARD** cancels a queued transaction.

- `WATCH key` can be used before `MULTI` for optimistic locking — if the watched key changes before `EXEC`, the transaction aborts.

`WATCH balance:1`

`MULTI`

`DECRBY balance:1 20`

`EXEC` # aborts (returns nil) if balance:1 changed since WATCH

11. Lua Scripting

Redis lets you run **Lua scripts** on the server itself. This matters because a script executes as a single atomic operation — no other command can run in the middle of it — and it saves round trips since the whole script runs server-side in one call.

Why use Lua instead of a transaction?

`MULTI/EXEC` batches commands atomically, but it can't make decisions — it can't check a value and conditionally do something different. A Lua script can read data, apply logic, and write results, all atomically and in one round trip.

Running a script with EVAL

`EVAL "return 'hello from Lua'" 0`

The `0` is the number of keys the script will operate on (see `KEYS/ARGV` below).

Using KEYS and ARGV

Scripts receive keys and arguments separately — this is a Redis convention so that Redis Cluster can figure out which node should run the script based on the keys involved.

`EVAL "return redis.call('SET', KEYS[1], ARGV[1])" 1 mykey "myvalue"`

- `KEYS[1]` → `mykey` (the first key argument)
- `ARGV[1]` → `"myvalue"` (the first extra argument)
- `1` → tells Redis there is 1 key argument

A more useful example — atomically increment a counter only if it's below a max:

```
-- script.lua

local current = tonumber(redis.call('GET', KEYS[1]) or "0")

if current < tonumber(ARGV[1]) then

    return redis.call('INCR', KEYS[1])

else

    return -1

end
```

Run it:

```
EVAL "$(cat script.lua)" 1 counter 100
```

EVALSHA — avoiding re-sending the script

Redis caches scripts by their SHA1 hash. Load a script once with **SCRIPT LOAD**, then invoke it repeatedly by hash — cheaper on the wire than sending the full script text each time.

```
SCRIPT LOAD "return redis.call('GET', KEYS[1])"
```

Returns a SHA1 hash, e.g. "e0e1f9fabfc9d4800c877a703b823ac0578ff831"

```
EVALSHA e0e1f9fabfc9d4800c877a703b823ac0578ff831 1 mykey
```

Common use cases for Lua scripts

- **Atomic check-and-set logic** (e.g., "only decrement stock if quantity > 0")
- **Custom rate limiters** (see the sliding-window example in the next section)
- **Distributed locks with safe release** (only delete a lock key if you're the one who set it)
- **Batch operations** that need to be all-or-nothing

Example: safely release a lock only if it belongs to you (**value** is a unique token you set when acquiring the lock):

```
if redis.call('GET', KEYS[1]) == ARGV[1] then

    return redis.call('DEL', KEYS[1])

else
```

```
return 0
```

```
end
```

```
EVAL "if redis.call('GET', KEYS[1]) == ARGV[1] then return redis.call('DEL', KEYS[1]) else  
return 0 end" 1 lock:job1 "my-unique-token"
```

This pattern avoids accidentally deleting a lock that another process has since acquired.

12. Pub/Sub Messaging

Redis supports simple publish/subscribe messaging for real-time communication between processes.

Terminal 1 (subscriber):

```
SUBSCRIBE news
```

Terminal 2 (publisher):

```
PUBLISH news "Breaking: Redis tutorial published!"
```

The subscriber instantly receives the message. You can also use pattern subscriptions:

```
PSUBSCRIBE news.*
```

```
PUBLISH news.tech "New tech article"
```

Note: Pub/Sub messages are fire-and-forget — if no one is subscribed when a message is published, it's lost. For durable messaging, look into **Redis Streams** ([XADD](#), [XREAD](#), [XGROUP](#)).

13. Persistence (RDB & AOF)

By default Redis is in-memory, but it offers two persistence options so data survives restarts:

RDB (Redis Database snapshots)

Periodic point-in-time snapshots of the dataset saved to disk.

In redis.conf

save 900 1 # save if at least 1 key changed in 900 seconds

save 300 10 # save if at least 10 keys changed in 300 seconds

Trigger manually:

SAVE # blocking save

BGSAVE # non-blocking, forks a background process

AOF (Append Only File)

Logs every write operation, replayed on restart. More durable than RDB, larger file size.

In redis.conf

appendonly yes

appendfsync everysec # options: always, everysec, no

Many production setups use both together for a balance of performance and durability.

14. Using Redis from Code (Python Example)

Install the client library:

pip install redis

Basic usage:

```
import redis
```

```
r = redis.Redis(host='localhost', port=6379, decode_responses=True)
```

```
# Strings
```

```
r.set('greeting', 'Hello from Python')
```

```
print(r.get('greeting'))

# Expiration

r.set('session:1', 'active', ex=60)

# Lists

r.rpush('queue', 'job1', 'job2')

print(r.lrange('queue', 0, -1))

# Hashes

r.hset('user:1', mapping={'name': 'Alice', 'age': 30})

print(r.hgetall('user:1'))

# Sorted sets

r.zadd('leaderboard', {'Alice': 100, 'Bob': 85})

print(r.zrevrange('leaderboard', 0, -1, withscores=True))

# Pipelines (batch commands for performance)

pipe = r.pipeline()

pipe.set('a', 1)

pipe.set('b', 2)

pipe.execute()
```

Similar client libraries exist for virtually every language: [node-redis](#) / [ioredis](#) (Node.js), [Jedis](#) / [Lettuce](#) (Java), [go-redis](#) (Go), [StackExchange.Redis](#) (.NET).

15. Common Use Cases

Use Case	How Redis Helps
Caching	Store expensive query/API results with a TTL to reduce load on your primary database
Session storage	Fast, shared session store across multiple app servers
Rate limiting	Use <code>INCR</code> + <code>EXPIRE</code> to count requests per user/IP within a time window
Leaderboards	Sorted sets naturally maintain ranked data
Job queues	Lists with <code>BLPOP/BRPOP</code> , or Streams for more robust queuing
Real-time analytics	HyperLogLog (<code>PFADD</code> , <code>PFCOUNT</code>) for approximate unique counts at scale
Geospatial queries	<code>GEOADD</code> , <code>GEODIST</code> , <code>GEOSEARCH</code> for location-based features
Distributed locks	<code>SET key value NX EX seconds</code> pattern for simple mutual exclusion

Rate Limiting in Depth

Rate limiting controls how many requests a user/IP/API key can make in a given time window. Redis is a natural fit because `INCR` and key expiration are both fast, atomic primitives.

1. Fixed window counter — simplest approach:

```
INCR rate:user123
```

```
EXPIRE rate:user123 60 # only needs to run once, when the key is first created
```

```
# If the returned count > limit, reject the request
```

The catch: `INCR` then `EXPIRE` is two separate commands, so there's a tiny gap where a crash could leave a key without a TTL. Fix it by setting the expiry only on the first increment:

```
SET rate:user123 0 EX 60 NX # creates the key with a 60s TTL only if it doesn't exist
```

INCR rate:user123

Fixed windows have a known edge-case flaw: a burst right at the boundary between two windows (e.g., 59 requests at the end of one minute, 59 more at the start of the next) can let through nearly double the intended limit.

2. Sliding window with a sorted set — more accurate, still simple:

On each request (pseudocode, run atomically via a script or pipeline):

```
ZADD rate:user123 <current_timestamp> <unique_request_id>
```

```
ZREMRANGEBYSCORE rate:user123 -inf (<current_timestamp> - 60000) # drop entries older than the window
```

```
ZCARD rate:user123 # count of requests remaining in the current window
```

```
EXPIRE rate:user123 60
```

If ZCARD result > limit, reject the request

This tracks exact request timestamps in a sorted set, trims anything outside the trailing window, and counts what's left — giving a true "last 60 seconds" limit rather than a fixed clock-aligned one.

3. Making it atomic with Lua — since the sliding window needs multiple commands to agree on one decision, wrap it in a script so no other request can interleave:

```
-- rate_limit.lua
```

```
local key = KEYS[1]
```

```
local now = tonumber(ARGV[1])
```

```
local window = tonumber(ARGV[2]) -- window size in ms
```

```
local limit = tonumber(ARGV[3])
```

```
redis.call('ZADD', key, now, now)
```

```
redis.call('ZREMRANGEBYSCORE', key, 0, now - window)
```

```
local count = redis.call('ZCARD', key)
```

```
redis.call('PEXPIRE', key, window)
```

```
if count > limit then
```

```
    return 0 -- reject
```

```
else
```

```
    return 1 -- allow
```

```
end
```

Call it:

```
EVAL "$(cat rate_limit.lua)" 1 rate:user123 1721990400000 60000 100
```

This guarantees the check-and-increment happens as one atomic step — no race condition between concurrent requests from the same user, which a plain sequence of client-side commands can't guarantee.

Choosing an approach:

Approach	Pros	Cons
Fixed window (INCR/EXPIRE)	Very simple, cheap	Allows bursts at window boundaries
Sliding window (sorted set)	Accurate, no boundary burst	More memory per user, slightly more commands
Sliding window + Lua	Accurate <i>and</i> atomic	Requires writing/maintaining a script

16. Replication & Clustering Basics

- **Replication:** A primary Redis instance can have one or more replicas that copy its data (**REPLICAOF host port**). Useful for read scaling and failover.
- **Redis Sentinel:** Monitors primary/replica setups and performs automatic failover if the primary goes down.

- **Redis Cluster:** Shards data automatically across multiple nodes using hash slots (16384 slots total), enabling horizontal scaling for large datasets.

These are typically configured at the infrastructure level rather than through everyday commands, and are worth exploring once you're comfortable with core Redis usage.

17. Security Basics

In redis.conf

requirepass your_strong_password_here

bind 127.0.0.1 -::1 # restrict network access

protected-mode yes

Connect with authentication:

redis-cli -a your_strong_password_here

Or from within a session:

AUTH your_strong_password_here

Also consider: running Redis behind a firewall, disabling dangerous commands (`rename-command FLUSHALL ""`), and using TLS for encrypted connections in newer Redis versions.

18. Performance Tips

- Avoid `KEYS *` in production — use `SCAN` instead, which iterates without blocking the server.
- Use pipelining to batch multiple commands and reduce round-trip latency.
- Set appropriate `maxmemory` and an eviction policy (e.g., `maxmemory-policy allkeys-lru`) so Redis doesn't run out of memory unexpectedly.
- Prefer hashes over many individual string keys when storing object-like data — it's more memory efficient.
- Monitor with `INFO`, `MONITOR` (careful — high overhead), and `SLOWLOG GET` to catch slow commands.

```
SCAN 0 MATCH user:* COUNT 100
```

19. Next Steps

Now that you've covered the fundamentals, consider exploring:

- **Redis Streams** — durable, consumer-group based messaging ([XADD](#), [XREADGROUP](#))
- **RedisJSON** and **RedisSearch** modules — for JSON documents and full-text search
- **Redis Cluster** — for horizontal scaling in production
- **Lua scripting** ([EVAL](#)) — for custom atomic operations
- Official docs: <https://redis.io/docs/latest/>

Practice by running through this tutorial in [redis-cli](#), then try wiring Redis into a small project — a caching layer, a rate limiter, or a simple leaderboard are all great starting points.